

*****Pornind de la aplicația exemplu pentru ciurul lui Eratostene din laboratorul 11, să se modifice aceasta astfel încât să se calculeze intersecția între A și B, unde A și B sunt două ADT-uri ce conțin fiecare câte 100 de valori inițializate aleator, apoi să se depună într-un ADT C rezultatele.

* La prb asta exista mai multe variante, cu reuniune si cu ceva suma *

LA VERSIUNEA CU REUNIUNE -- inlocuim cuvantul intersection cu Union in cod ---
(se modifica 1 singur cuvant in tot codul + denumirile functiilor/claselor)

PT versiunea cu suma IMPROVISE ADAPT OVERCOME
LAB11
DESCHID DIN ZIP UL LABORATORULUI ERATOSTE-MAVEN
Fisierul POM.XML ii nemodificat
si modific fisierele care is acolo, unu am impresia ca ii in plus
Vezi ca ii ceva cu MICRONAUT AI GRIJA LA INSTALURI AND SHIT
**** GASIT ***

```
-----  
Application.kt  
  
package com.sd.laborator  
  
import io.micronaut.http.annotation.Body  
import io.micronaut.http.annotation.Controller  
import io.micronaut.http.annotation.Post  
import io.micronaut.runtime.Micronaut  
  
object Application {  
    @JvmStatic  
    fun main(args: Array<String>) {  
        Micronaut.run(Application::class.java, *args)  
    }  
  
    @Controller  
    class LambdaController {  
        @Post  
        fun execute(@Body request: IntersectionRequest): IntersectionResponse {  
            return handler.apply(request)  
        }  
  
        companion object {  
            private val handler = IntersectionFunction()  
        }  
    }  
}
```

```

    }
  }
}

```

 ..Function.kt

```
package com.sd.laborator;
```

```
import io.micronaut.function.executor.FunctionInitializer
import org.slf4j.Logger
import org.slf4j.LoggerFactory
import java.util.function.Function
```

```
class IntersectionFunction : FunctionInitializer(), Function<IntersectionRequest,
IntersectionResponse> {
    private val LOG: Logger = LoggerFactory.getLogger(IntersectionFunction::class.java)
```

```

    override fun apply(msg : IntersectionRequest) : IntersectionResponse {
        LOG.info("Transform A intr-un set...")
        val A = msg.getA().toSet()
        LOG.info("Transform B intr-un set...")
        val B = msg.getB().toSet()
        LOG.info("Calculati intersectia lui A cu B in C...")
        val C = A intersect B
        LOG.info("Creat raspunsul...")
        val response = IntersectionResponse()
        response.setC(C.toList())
        response.setMessage("OK!")
        LOG.info("Cerere tratata!")
        return response
    }
}

```

```
/**
```

```

 * This main method allows running the function as a CLI application using: echo '{}' | java -jar
function.jar

```

```

 * where the argument to echo is the JSON to be parsed.

```

```
*/
```

```
fun main(args : Array<String>) {
    val function = IntersectionFunction()
    function.run(args, { context -> function.apply(context.get(IntersectionRequest::class.java))})
}

```

..Request.kt

```
package com.sd.laborator
```

```
import io.micronaut.core.annotation.Introspected
```

```
@Introspected
```

```
class IntersectionRequest {  
    private lateinit var a: List<Int>  
    private lateinit var b: List<Int>
```

```
    fun getA(): List<Int> = a
```

```
    fun getB(): List<Int> = b
```

```
}
```

...Response.kt

```
package com.sd.laborator
```

```
import io.micronaut.core.annotation.Introspected
```

```
@Introspected
```

```
class IntersectionResponse {  
    private var message: String? = null  
    private var c: List<Int>? = null
```

```
    fun getC(): List<Int>? = c
```

```
    fun setC(C: List<Int>?) {  
        this.c = C  
    }
```

```
    fun getMessage(): String? = message
```

```
    fun setMessage(message: String?) {  
        this.message = message  
    }
```

```
}
```